

SLT INTERN AGENTIC AI

Project Documentation Report

A Question-Routing Agent over PDF Documentation and a MySQL Product Database

1. Introduction

This document describes the design, implementation, and reasoning behind an agentic AI application built during the SLT internship. The system answers natural-language questions about a fictional electronics brand, TechNova, by deciding — on its own, per question — whether to retrieve the answer from a PDF product manual or from a live MySQL product database, and sometimes from both. The purpose of the project was to explore how large language models can be extended beyond text generation into systems that reason about which data source to consult and combine results into a single, coherent answer.

The project intentionally combines two very different kinds of data retrieval in one agent: unstructured document search (a PDF manual, searched by meaning rather than keyword) and structured database lookup (exact prices and stock counts). This mirrors a common real-world enterprise pattern, where some information lives in static documentation and other information changes constantly in a live database.

1.1 Purpose of this Document

This document is intended to explain, from first principles, what tools and components were used to build the system, why each was chosen over its alternatives, how the pieces operate together at runtime, and what would need to be added before this prototype could be considered production-ready.

2. Technology Stack and Justification

Every tool used in this project was chosen deliberately. The table below summarises each component, its purpose, and the reasoning behind selecting it over the available alternatives.

Component	Purpose	Reason Selected
Groq API (openai/gpt-oss-120b)	Hosts the large language model that powers the agent's reasoning and tool-calling.	Groq offers free-tier access with very low inference latency. The originally used model, llama-3.3-70b-versatile, was deprecated by Groq in June 2026, so the project was migrated to openai/gpt-oss-120b, Groq's recommended replacement with more reliable structured tool-calling.
LangChain	Provides the abstraction layer for chat models, prompts, and tool integration.	Widely adopted, well-documented, and integrates directly with both Groq and the Model Context Protocol without custom glue code.
LangGraph (create_react_agent)	Implements the ReAct reasoning loop that lets the model call tools, observe results, and decide on further actions.	Provides a production-tested implementation of the reasoning loop rather than requiring one to be written from scratch.
MCP (Model Context Protocol) / FastMCP	Exposes each data source (PDF search, database queries) as an independent tool server the agent can call.	Decouples tool implementation from agent logic. Each data source can be developed, tested, and restarted independently of the LLM layer.
Chroma	Vector store used to persist and search PDF chunk embeddings.	Simpler installation than FAISS (pure Python, no compiled binary dependencies), which matters on Windows where FAISS occasionally has build issues. For a small,

Component	Purpose	Reason Selected
		single-document project, Chroma's slightly lower raw throughput versus FAISS is irrelevant.
sentence-transformers (all-MiniLM-L6-v2)	Converts text into numeric vector embeddings for semantic search.	Runs entirely locally with no API key or per-call cost, and is small enough (~90MB) to download and run on a typical laptop.
MySQL	Stores structured, frequently-changing product data (price, stock, category).	Chosen as the relational alternative to MongoDB for this iteration of the project. MySQL is widely taught, widely deployed, and well-suited to the fixed, tabular schema this data naturally has (a fixed set of columns per product).
Python (pymongo replaced by mysql-connector-python)	Client library for connecting the tool server to the database.	Official MySQL driver maintained by Oracle with direct parameterised-query support to prevent SQL injection.

2.1 Why Not a Single Combined Tool?

An alternative design would combine PDF search and database lookup into one tool and let application code decide which source to query, using string matching on the question (for example, checking whether the word 'price' appears). This project deliberately avoids that approach. Keeping the tools separate and letting the language model choose between them, based only on each tool's description, is what makes the system agentic rather than a scripted decision tree. It also means the system degrades gracefully: a new tool can be added later (for example, a shipping-tracking API) without rewriting any routing logic, since the model reads tool descriptions at runtime.

3. System Architecture

The diagram below shows the complete request lifecycle: a user's question is passed to the agent, which reasons about which tool or tools are relevant, retrieves data from the corresponding source, and synthesises a final answer.

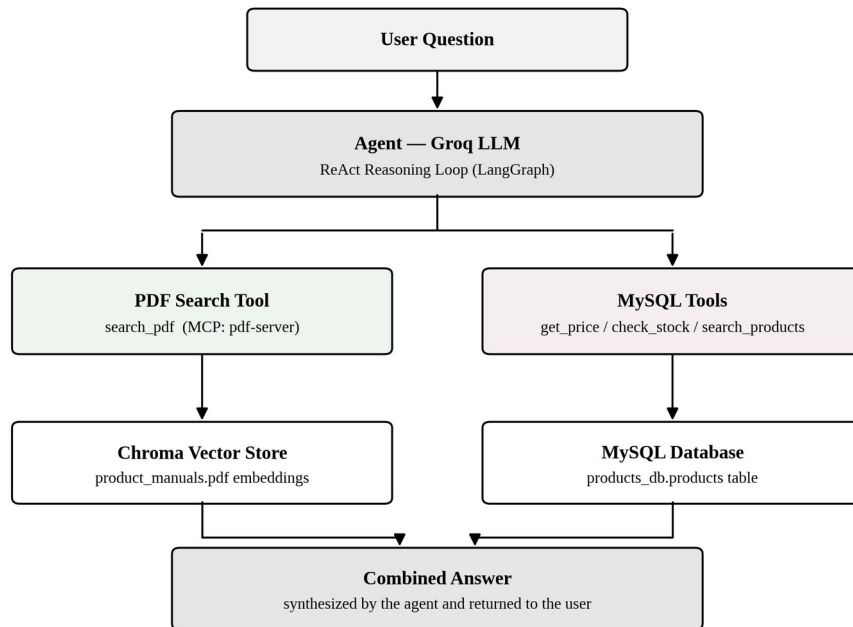


Figure 1: High-level architecture showing question routing between the PDF search tool and the MySQL tools.

No conditional logic anywhere in the codebase decides which tool to call. Instead, each tool exposed through MCP carries a natural-language description of exactly what kind of question it should be used for (for example, 'use this for warranty, specifications, or troubleshooting questions — not for prices or stock'). The language model reads these descriptions at runtime, alongside the user's question, and decides which tool call(s), if any, are appropriate. This is the mechanism referred to throughout this document as tool-description-driven routing.

4. How PDF Retrieval Works (Retrieval-Augmented Generation)

The PDF side of the system implements a standard Retrieval-Augmented Generation (RAG) pattern, split into two distinct phases: an offline ingestion phase that runs once, and an online retrieval phase that runs on every question.

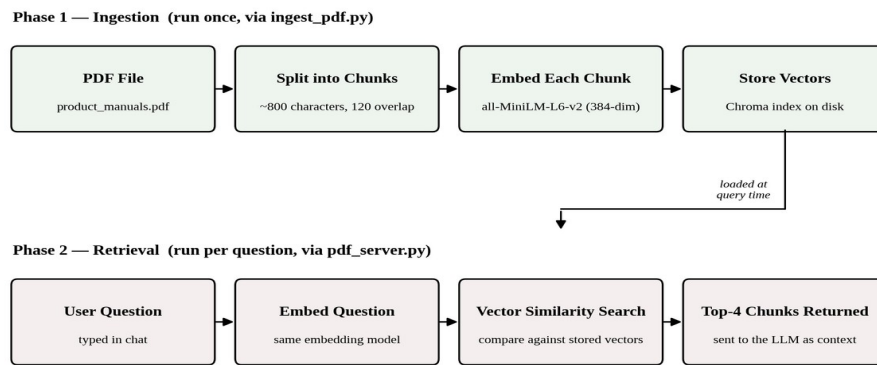


Figure 2: The two-phase PDF pipeline — ingestion (top) and retrieval (bottom).

4.1 Ingestion Phase (`ingest_pdf.py`)

- The PDF is loaded page by page using `PyPDFLoader`.
- Each page is split into overlapping chunks of approximately 800 characters, with 120 characters of overlap, so that sentences near a chunk boundary are not stripped of their surrounding context.
- Each chunk is converted into a 384-dimensional numeric vector using the `all-MiniLM-L6-v2` embedding model, which runs locally.
- All vectors are stored in a Chroma index persisted to disk, so ingestion does not need to be repeated unless the source PDF changes.

4.2 Retrieval Phase (`mcp_servers/pdf_server.py`)

- The user's question is embedded using the same model used during ingestion, so both the question and the document chunks live in the same vector space.
- Chroma performs a similarity search (cosine distance) to find the four stored chunks whose vectors are closest to the question's vector.
- The matching chunks are returned as plain text, tagged with their source file and page number, and handed to the language model as context for its final answer.

Because retrieval is based on vector distance rather than keyword overlap, a question such as 'how long is Laptop A covered' can correctly match a chunk that says '2-year limited warranty', even though no words are shared between the question and the answer.

5. How Structured Data Retrieval Works (MySQL)

Unlike the PDF path, product price, stock, and category data does not benefit from semantic search: these are exact, structured values that should be matched precisely rather than by meaning. For this reason, the MySQL side of the system uses conventional parameterised SQL queries rather than embeddings.

- `get_price(product_name)` — runs a `SELECT` with a `LIKE` clause to find the current price of a named product.
- `check_stock(product_name)` — returns the current stock count and an in-stock/out-of-stock status.

- `search_products(query)` — searches by partial product name or category, returning a formatted list of matches.

All queries use parameterised placeholders (`%s`) rather than string concatenation, which prevents SQL injection regardless of what a user or the language model passes as a search term. This claim is verified directly in Section 8.4.

6. Project Component Map

The diagram below shows how the project's individual files depend on one another, from configuration through to the two user-facing entry points.

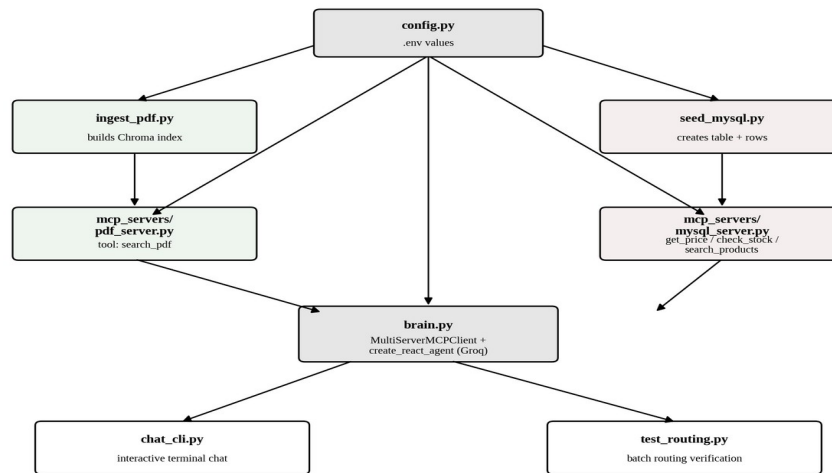


Figure 3: Dependency relationships between the project's source files.

File	Role
config.py	Loads all configuration (API keys, database credentials, file paths) from a .env file into one place, so no secrets are hardcoded elsewhere.
ingest_pdf.py	One-time script that builds the Chroma vector index from the PDF manual.
seed_mysql.py	One-time script that creates the MySQL database, table, and inserts sample product rows.
mcp_servers/pdf_server.py	MCP tool server exposing search_pdf, backed by the Chroma index.
mcp_servers/mysql_server.py	MCP tool server exposing get_price, check_stock, and search_products, backed by MySQL.
brain.py	Connects to both MCP servers, retrieves their tools, and assembles the Grog-powered ReAct agent.
chat_cli.py	Interactive terminal chat interface for talking to the agent directly.
test_routing.py / test_boundary.py / test_description_length.py	Automated test scripts covering baseline routing, edge-case/failure behaviour and tool-description-length experiments. Detailed in Section 8.

7. Agent Reasoning and Routing Logic

The agent is built with LangGraph's `create_react_agent` function, which implements the ReAct (Reason and Act) pattern: the model alternates between reasoning about what it needs and calling a tool to get it, repeating until it has enough information to answer.

- The user's question, along with a system prompt describing the two categories of tools, is sent to the model.
- The model decides whether to call `search_pdf`, one or more MySQL tools, both, or neither.

- If a tool is called, its result is added to the conversation and the model reasons again — it may call a second tool if the question required information from both sources.
- Once the model has sufficient information, it produces a final natural-language answer synthesising whatever was retrieved.

A system prompt inside `brain.py` explicitly instructs the model to call both tools when a question needs both kinds of information, and to state honestly when neither tool returns a relevant result, rather than guessing.

8. Testing and Optimisation

Following supervisor review, the system was subjected to systematic testing across four areas: baseline routing correctness, tool-description-length sensitivity, system boundary and failure-mode behaviour, and the source of response latency. This section reports the actual measured results, not projected or assumed behaviour.

8.1 Baseline Routing Verification

A fixed battery of eight questions was run against the live agent (`test_routing.py`), covering single-tool questions for both the PDF and MySQL sides, a question requiring both tools simultaneously, and a category-browsing question. All eight questions routed correctly, including the dual-tool case ('What is the price and warranty of Laptop A?'), which correctly triggered both `get_price` and `search_pdf` and combined their results into one answer. This confirmed the core routing mechanism was functioning correctly before more adversarial testing began.

8.2 Tool Description Length Experiment

To answer the question of how long a tool's description should be, the same underlying PDF search tool was exposed to the agent with three different description lengths, and an identical set of test questions was run against each variant (`test_description_length.py`). The measured results were:

Variant	Length (words)	Accuracy	Avg. Latency
Too short	2	60%	4.18s
Reasonable	25	80%	4.91s
Too long / verbose	145	100%	6.44s

An important nuance qualifies these results. In this experiment, only one tool was available to the agent, so a 'misroute' did not mean the wrong tool was called — it meant no tool was called at all, with the model answering from its own assumption instead of retrieving real data. The finding is therefore best stated precisely: a longer, more explicit description made the model more confident that borderline questions (for example, 'how do I maximise battery life', which could sound like general advice) were in fact covered by the tool, at a measurable latency cost of approximately 54% between the shortest and longest variants tested. This experiment did not test whether an overly long description could cause confusion when multiple competing tools are available — that remains an open question for future testing.

Recommendation: tool descriptions should be closer to the 'too long' variant in explicitness for coverage-critical tools, but the accuracy/latency trade-off should be revisited once a mid-length variant (approximately 60–80 words) is tested to locate a better balance point.

8.3 System Boundary and Failure-Mode Testing

Thirteen deliberately adversarial inputs were run against the live agent (`test_boundary.py`), covering empty input, gibberish, non-English input, an extremely long repeated input, questions with no matching data, SQL-injection-shaped strings, a prompt-injection attempt, an irrelevant general-knowledge question, and mixed answerable/unanswerable questions.

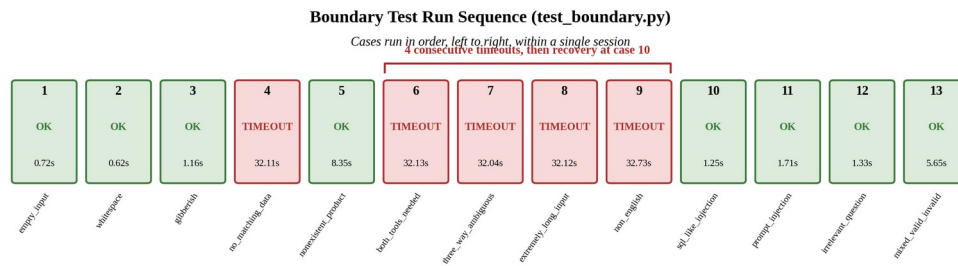


Figure 4: Run sequence of the 13 boundary test cases, showing the cluster of consecutive timeouts.

Eight of the thirteen cases behaved correctly: empty and whitespace-only input were handled gracefully with a clarifying reply rather than a crash; a non-existent product correctly produced a 'not found' response rather than a fabricated stock number; a prompt-injection attempt was refused outright; and a mixed question (one answerable part, one not) was correctly split, answering the resolvable half honestly while flagging the other as unavailable.

Five of the thirteen cases — questions with no matching data, a dual-tool question, a multi-fact comparison question, an extremely long repeated input, and a non-English question — exceeded the script's 30-second timeout. The single most important diagnostic finding from this round of testing is that one of these cases, the dual-tool question 'What's the price and warranty of Laptop A?', is textually identical to a question that succeeded cleanly during the baseline routing test in Section 8.1. The same question therefore produced two different outcomes on two different runs, which rules out an explanation based on the question itself being inherently unanswerable or too complex. The failure is intermittent and environment-dependent rather than a fixed limitation of that query type.

The run sequence in Figure 4 supports this conclusion further: the five timeouts are not scattered randomly through the thirteen cases, but occur mostly as a cluster of four consecutive failures (cases 6 through 9), immediately preceded and followed by successful runs. A permanent, architectural limitation would be expected to fail consistently regardless of position in the run; a cluster with clean recovery on either side is more consistent with a temporary condition affecting the whole session for a window of time.

Two hypotheses were identified as the most likely explanation, to be distinguished by further testing:

- Groq API rate limiting — the free tier enforces requests-per-minute and tokens-per-minute caps, which could plausibly be reached after running many LLM calls in quick succession across a test session, causing calls to stall or silently retry.
- An uncapped ReAct reasoning loop — the agent currently has no explicit limit on how many reasoning-and-tool-call rounds it may take before answering. A more ambiguous question may require several rounds, each involving a full LLM round-trip of several seconds; enough rounds could plausibly accumulate past the 30-second ceiling even without any external rate limiting.

A follow-up script (test_timeout_isolation.py) was created to distinguish between these two explanations directly: it re-runs only the five timed-out questions in isolation, spaced several seconds apart with a longer timeout, and records how many reasoning rounds each question required. If the questions now succeed consistently, the original timeouts were caused by session-level load or rate

limiting; if they still time out with a high round count, the agent requires an explicit recursion or step limit. This isolation test is the immediate next action item arising from this round of testing.

8.4 Security Boundary: SQL Injection Verification

The SQL-injection-shaped boundary case in the adversarial test battery did not, in itself, prove the system was safe against injection: the model chose not to call any database tool for that particular input, so the malicious string was never actually passed to the database layer in that run. To properly verify this property, a direct test (`test_sql_injection_direct.py`) was created that calls the exact query pattern used in `get_price()` with several injection payloads — including a DROP TABLE attempt — bypassing the language model entirely, and checks the row count of the products table before and after. Because every query in the system uses parameterised placeholders (`%s`) rather than string concatenation, an injection payload is treated as a literal search string rather than executable SQL, and the table's row count is expected to remain unchanged across all payloads. This distinction — testing the LLM's behaviour versus testing the underlying code's safety directly — is an important methodological point: agentic systems should have their tool implementations verified independently of whether the model happens to choose to call them during any given test run.

8.5 Secondary Findings

- An irrelevant general-knowledge question ('What is the capital of France?') was answered directly and correctly rather than being flagged as out of scope. This is not a bug — the system prompt does not currently instruct the agent to restrict itself to TechNova-related topics — but it is a product decision that should be made explicitly: whether the assistant should answer general questions or redirect all off-topic queries.
- A deprecation warning was observed during testing: LangGraph's `create_react_agent` has been moved to `langchain.agents.create_agent` and is scheduled for removal in LangGraph v2.0. This is a near-term maintenance item independent of the performance findings above.
- The 'nonexistent_product' boundary case correctly reported 'not found', but did so by calling two tools in sequence (`check_stock`, then `search_products`) rather than one, adding avoidable latency to what should be a simple negative-result response.

8.6 Summary of Optimisation Recommendations

- Add an explicit recursion/step limit to the ReAct agent so that an ambiguous question cannot silently consume the full client-side timeout through repeated reasoning rounds.
- Investigate Groq API usage against rate-limit thresholds during test sessions; consider adding exponential backoff and explicit handling of rate-limit responses rather than allowing calls to stall silently.
- Pool MySQL connections rather than opening a new connection per tool call, as each of `get_price`, `check_stock`, and `search_products` currently does independently.
- Settle on a tool description length in the 60–80 word range as a starting point, balancing the accuracy gains observed at 145 words against the latency cost, and re-test once a competing second tool is included in the same experiment.
- Decide and document explicitly whether the agent should answer general, off-topic questions or restrict itself to product-related queries only.
- Plan a migration from `create_react_agent` to `langchain.agents.create_agent` ahead of the LangGraph v2.0 removal.

9. Moving to Production: What Needs to Be Added

The current system is a functioning, tested prototype, not a production system. The sections below outline what would need to be added, grouped by concern, before this could be deployed for real users or real business data. Several items here are reinforced directly by the testing findings in Section 8, rather than being purely theoretical concerns.

9.1 Security

- Replace .env-based secrets with a managed secrets store (for example, AWS Secrets Manager, Azure Key Vault, or HashiCorp Vault), so credentials are never stored in plaintext files on disk.
- Add authentication and authorisation to the chat interface itself — the current CLI has no concept of a logged-in user.
- Add input validation and rate limiting on incoming questions to prevent abuse and excessive API cost.
- Add defences against prompt injection, particularly if PDF content or database records could ever contain untrusted, user-supplied text.
- Restrict the MySQL database user's privileges to only what the application tools require (SELECT only, for example), rather than a broadly privileged account.
- Enforce TLS/SSL on the MySQL connection and restrict network access to known application servers only.

9.2 Reliability and Error Handling

- Add retry logic with exponential backoff around calls to the Groq API and the database, since transient failures are inevitable in production — directly motivated by the intermittent timeouts observed in Section 8.3.
- Add a fallback model or provider in case the primary model is deprecated, rate-limited, or experiencing an outage — the project's own experience with Groq's model deprecation demonstrates this is not a theoretical risk.
- Add explicit timeouts and a bounded reasoning-step limit on every external call and on the agent loop itself, so a single slow dependency or an ambiguous question cannot hang the entire system.
- Add health-check endpoints for each MCP server so failures can be detected automatically rather than discovered by a failing user request.

9.3 Scalability

- Move from stdio-based MCP servers (single local subprocess per session) to persistently running, independently scalable MCP services, so multiple concurrent users do not each spin up new server processes.
- Introduce a connection pool for MySQL rather than opening a new connection per tool call.
- Containerise each component (Docker) so the agent, PDF tool server, and database tool server can be deployed, scaled, and updated independently.
- Consider a managed or clustered vector database (for example, a hosted Chroma deployment, Pinecone, or Weaviate) if the document collection grows beyond a single small PDF.
- Add caching for frequently repeated questions or frequently retrieved chunks, to reduce redundant LLM and database calls.

9.4 Observability

- Add structured logging of every tool call, including which tool was selected, its arguments, and its result, to support debugging routing issues in production.
- Add tracing (for example, OpenTelemetry) across the full request lifecycle, from user question through tool calls to final answer — this would have made diagnosing the Section 8.3 timeout cluster immediate rather than requiring a follow-up isolation test.
- Add metrics and alerting on latency, error rate, and tool-selection distribution, so misrouting or degraded model behaviour is caught automatically rather than through manual testing.
- Track token usage and cost per conversation to monitor spend as usage scales.

9.5 Data Management

- Establish a proper document ingestion pipeline that can detect when the source PDF (or a growing set of PDFs) has changed and automatically re-run ingestion, rather than requiring a manual script run.
- Add automated backups and a disaster-recovery plan for the MySQL database.
- Introduce schema migrations (for example, using a tool such as Alembic or Flyway) rather than a script that drops and recreates tables.
- Version the vector index alongside the source documents, so the deployed index and the documents it was built from can always be traced back to one another.

9.6 Testing and Quality Assurance

- Expand the current test scripts into an automated suite that runs in continuous integration on every code change, with a larger and more adversarial set of test questions than the thirteen used in Section 8.3.
- Add unit tests for each MCP tool in isolation, independent of the language model, following the same approach used for the SQL injection verification in Section 8.4.
- Add an evaluation framework that scores answer quality, not just which tool was called, since a correct tool choice does not guarantee a correct final answer.
- Add a staging environment that mirrors production configuration for safe pre-release testing.

9.7 User Experience and Interface

- Replace the terminal-based CLI with a proper web or chat-application front end suitable for non-technical end users.
- Add conversation/session management so multiple users can hold independent conversations concurrently.
- Add graceful, user-facing error messages when a tool or the model is unavailable, rather than surfacing raw stack traces or an unexplained hang.

9.8 Compliance and Governance

- Review what categories of data may end up in the PDF documentation or product database, and ensure handling complies with applicable data-protection regulations.
- Establish a content-moderation or guardrail layer if the system will ever be exposed to the public internet.
- Define clear data-retention and audit-logging policies for conversations, particularly if customer-identifiable information is ever involved.